



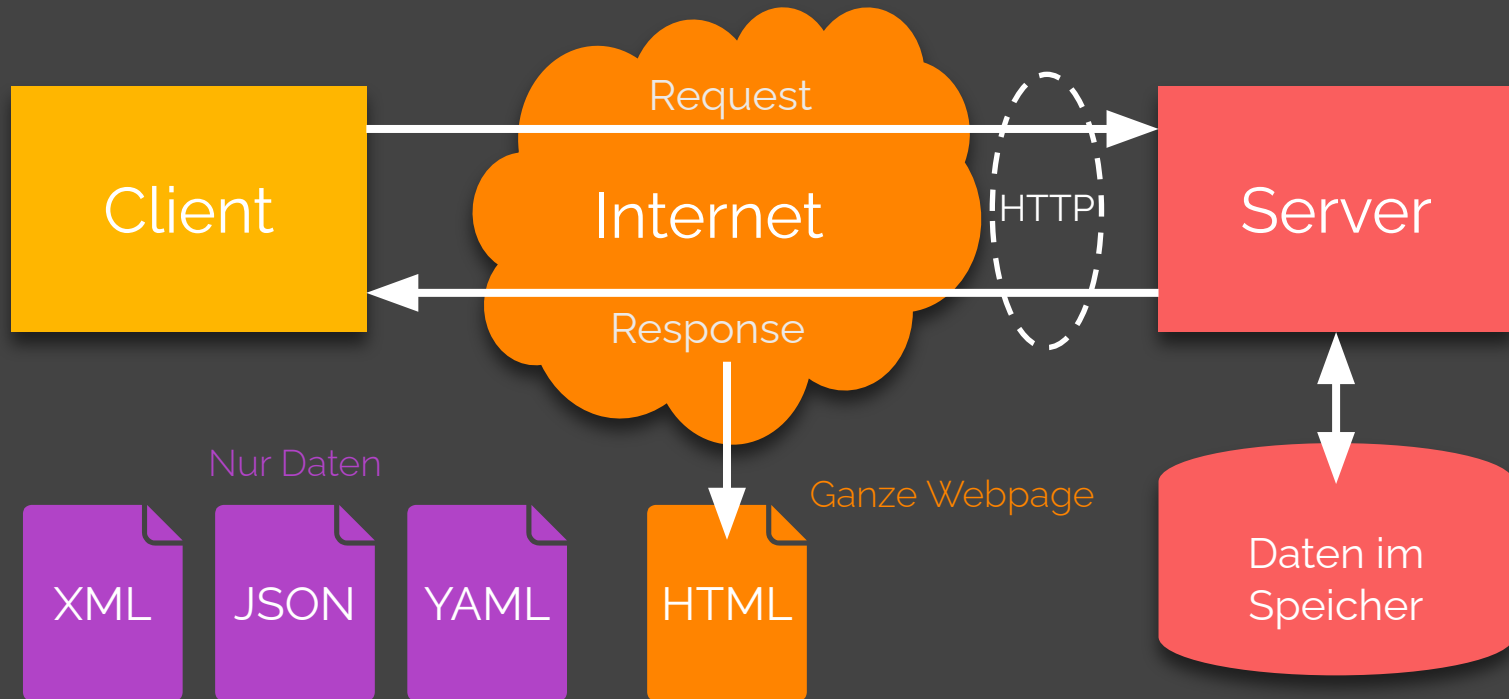
Web Fundamentals

WEF1VO

Teil 7: Datenformate

Medientechnik und -design | WS 2025/26

Daten übertragen ?



Daten speichern ?



Der klassische Tiroler Knödel

- 350 g Semmelwürfel
 - 200 ml Milch
 - 4 Stück Eier
 - 100 g Speck
 - 100 g Bergsteigerwurst
 - 20 g Butter
 - 1 Stück Zwiebel
 - 1 TL Petersilie
 - 1 Prise Salz
 - 1 Prise Muskatnuss
 - 80 g Mehl
 - 10 g Schnittlauch
1. Semmelwürfel mit Milch und Eiern vermengen und 20 bis 30 Minuten ziehen lassen.
 2. Kleinwürfelig geschnittenen Speck und fein gewiegte Bergsteigerwurst bzw. Kaminwürsten in zerlassener Butter gemeinsam mit Zwiebeln, Petersilie und Schnittlauch anbraten und mit Salz und Muskatnuss abschmecken. Mehl darüber streuen und aus allen Zutaten eine feste, aber nicht zu feuchte Masse kneten.
 3. Mit befeuchteten Händen daraus Knödel formen.
 4. Einen davon als Probeknödel in Salzwasser knappe 15 Minuten kochen. Hat der Probeknödel eine zu weiche Konsistenz, muss man noch etwas Mehl unter die Knödelmasse kneten.
 5. Erst wenn die Masse kompakt genug ist, kocht man die restlichen Knödel.
 6. Tiroler Knödel mit Schnittlauch bestreut servieren.

Agatha

11

**startet in
Richtung
Mond**



Landung auf dem Mond





**Allerhand Daten
werden
aufgezeichnet**



Heute ist
viele
davon
nicht mehr
lesbar

Textbasiert e Daten- formate!

Textbasierte Datenformate sind menschen-
und maschinenlesbar!



XML

Die Extensible Markup Language



Was XML ist...

Meta-Markupsprache

Eine Markupsprache, mit der Markupsprachen definiert werden können.

Untermenge der Standardized General Markup Language mit einer strengen Grammatik.

Datenformat

XML-Dateien sind Textdokumente.

Eine XML-Anwendung ist eine Menge an definierten Tags und Attributen.



Ein Rezept für **Tiroler Knödel** in XML

Ein Rezept strukturiert (menschen- und maschinenlesbar) mit Hilfe von XML darstellen.



Rezept Teil 1



```
<?xml version="1.0" encoding="UTF-8"?>
<rezept quelle="https://www.ichkoche.at/der-klassische-tiroler-knoedel-rezept-4386">
  <gericht>Der klassische Tiroler Knödel</gericht>
  <zutaten>
    <zutat>
      <ingredient>Knödelbrot</ingredient>
      <menge>350</menge>
      <einheit>g</einheit>
    </zutat>
    <zutat>
      <ingredient>Milch</ingredient>
      <menge>200</menge>
      <einheit>ml</einheit>
    </zutat>
    ...
  </zutaten>
  ...
</rezept>
```

Rezept Teil 2



...

<zubereitung>

<schritt>Semmelwürfel mit Milch und Eiern vermengen und 20 bis 30 Minuten ziehen lassen.</schritt>

<schritt>Kleinwürfelig geschnittenen Speck und fein gewiegte Bergsteigerwurst in zerlassener Butter gemeinsam mit Zwiebeln, Petersilie und Schnittlauch anbraten und mit Salz und Muskatnuss abschmecken. Mehl darüber streuen und aus allen Zutaten eine feste, aber nicht zu feuchte Masse kneten.</schritt>

<schritt>Mit befeuchteten Händen daraus Knödel formen.</schritt>

<schritt>Einen davon als Probeknödel in Salzwasser knappe 15 Minuten kochen. Hat der Probeknödel eine zu weiche Konsistenz, muss man noch etwas Mehl unter die Knödelmasse kneten.</schritt>

<schritt>Erst wenn die Masse kompakt genug ist, kocht man die restlichen Knödel.</schritt>

<schritt>Tiroler Knödel mit Schnittlauch bestreut servieren.</schritt>

</zubereitung>

</rezept>



**Well-formed
& valid**

700

Well- formedness



Die XML-Grammatikregeln einhalten.



Basic Rules

Ein Wurzelement

Attribute in
Anführungszeichen,
müssen einzigartig sein

```
<rezept quelle="...">  
  <gericht>...</gericht>  
  <zutaten>...</zutaten>  
  <zubereitung>...</zubereitung>  
</rezept>
```

Paarweise Tags Keine Überkreuzung

Elemente ohne Inhalt:

```
<leer></leer>
```

Oder besser:

```
<leer />
```



Namen in XML

Well-formed (korrekt):

```
<Drivers_License_Nr>98NY32</Drivers_License_Nr>
```

```
<month-day-year>7/23/2001</month-day-year>
```

```
<first_name>Alan</first_name>
```

```
<_4-lane>I-610</_4-lane>
```

```
<téLéphone>011 33 91 55 27 55 27</téLéphone>
```

Nicht well-formed (inkorrekt):

```
<Driver's_License_Number>98NY32</Driver's_License_Number> (Apostroph)
```

```
<month/day/year>7/23/2001</month/day/year> (Schrägstrich)
```

```
<first name>Alan</first name> (Leerzeichen)
```

```
<4-lane>I-610</4-lane> (beginnt mit Zahl)
```



Entitäten , PCDATA , CDATA

5 Entitäten, die immer ersetzt werden müssen:

<	>	&	"	'
<	>	&	"	'

```
<element>  
  <name><![CDATA[<h1>]]></name>  
  <description>Heading Level 1</description>  
  <example><![CDATA[<h1>Hello World</h1>]]></example>  
</element>
```

CDATA:
Character Data
(statt <h1>);

PCDATA:
Parsed Character Data



Kommentare , PI und Präfix

XML-Kommentar

```
<!-- Das Root-Element -->
```

Processing Instruction

```
<?robots index="yes" follow="no"?>
```

XML-Deklaration/Präfix

```
<?xml version="1.0" encoding="UTF-8"?>
```

Validität

Einer Document Type Definition (DTD) oder einem Schema entsprechen.



DTDs

Document Type Definitions (DOCTYPE)





Aufbau einer DTD

Textdokument (Erweiterung `.dtd`) mit Deklarationen im Format

`<!TYP Detailangaben>`

Elemente

Der Typ `ELEMENT` deklariert **Elemente**, d.h. Tags.

Attribute

Der Typ `ATTLIST` deklariert ein **Attribut** zu einem Element.

Entitäten

Der Typ `ENTITY` deklariert eine **Entität**, d.h. eine Art Konstante.

Elemente deklarieren



```
<!ELEMENT title (content)>
```

```
<!ELEMENT foo (EMPTY)>
```

```
<!ELEMENT anmerkung (#PCDATA)>
```

```
<!ELEMENT random (ANY)>
```

```
<!ELEMENT familie (kind)>
```

```
<!ELEMENT auto (hersteller, typ, baujahr)>
```

```
<!ELEMENT sport (fußball|tennis|laufen)>
```

Anzahl festlegen



? 0 oder 1 Mal

+ 1 bis n Mal

* 0 bis n Mal

() Gruppierung

<!ELEMENT Nachricht

(absender, empfänger+, inhalt, (bcrypt|argon2i)?)>

Attribute deklarieren



```
<!ATTLIST tag attr type default>
```

```
<!ATTLIST song title CDATA #REQUIRED>
```

```
<!ATTLIST vehikel typ  
  (pkw|lkw|flugzeug|boot) "pkw">
```

Tiroler Knödel **deklariert**



```
<!ELEMENT rezept (gericht, zutaten, zubereitung)>
<!ATTLIST rezept quelle CDATA #REQUIRED>
<!ELEMENT gericht (#PCDATA)>
<!ELEMENT zutaten (zutat)+>
<!ELEMENT zubereitung (schritt)+>
<!ELEMENT zutat (ingredienz, menge, einheit)>
<!ELEMENT schritt (#PCDATA)>
<!ELEMENT ingredienz (#PCDATA)>
<!ELEMENT menge (#PCDATA)>
<!ELEMENT einheit (#PCDATA)>
```

DTDs einbinden



```
<!DOCTYPE name SYSTEM url>
```

```
<!DOCTYPE rezept SYSTEM  
  "https://www.hochleitner.cc/dtds/rezept.dtd">
```

```
<!DOCTYPE html  
  PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"  
  "http://www.w3.org/TR/xhtml1/DTD/  
  xhtml1-strict.dtd">
```


XSD

XML Schema Definition





Aufbau einer XSD

Textdokument (Erweiterung `.xsd`) mit Deklarationen im XML-Format:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">  
...  
</xs:schema>
```

Elemente

Der Tag `<xs:element>`
deklariert **Elemente**, d.h.
Tags.

Attribute

Der Tag `<xs:attribute>`
deklariert ein **Attribut**.

Komplexe Typen

Der Tag
`<xs:complexType>`
deklariert eine **komplexe
Typen**, d.h. Elemente mit
weiteren Elementen.

Elemente deklarieren



```
<xs:element name="..." type="..." />
```

```
<xs:element name="gericht" type="xs:string"/>
```

```
<xs:element name="menge" type="xs:decimal"/>
```

```
<xs:element name="zutat">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="ingredienz" type="xs:string"/>
      <xs:element name="menge" type="xs:decimal"/>
      <xs:element name="einheit" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Anzahl festlegen



`minOccurs="0"`

Optional

`maxOccurs="unbounded"`

Beliebig oft

`minOccurs="1"`

Mindestens einmal.

```
<xs:element name="schritt"
  type="xs:string"
  maxOccurs="unbounded" />
```

Attribute deklarieren



```
<xs:attribute name="..." type="..." use="..." />
```

type: all, decimal, date, ID, string, ...

use: required, optional, prohibited

```
<xs:attribute name="quelle"  
              type="xs:anyURI"  
              use="required" />
```

Tiroler Knödel **deklariert**



```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="rezept">
    <xs:complexType>
      <xs:sequence>
        <xs:element ref="gericht"/>
        <xs:element ref="zutaten"/>
        <xs:element ref="zubereitung"/>
      </xs:sequence>
      <xs:attribute name="quelle" type="xs:anyURI" use="required"/>
    </xs:complexType>
  </xs:element>

  <xs:element name="gericht" type="xs:string"/>

  <xs:element name="zutaten">
    <xs:complexType>
      <xs:sequence>
        <xs:element maxOccurs="unbounded" ref="zutat"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
```

Verweis auf die
Deklaration an anderer
Stelle (reduziert
Verschachtelung)

Tiroler Knödel **deklariert**



```
<xs:element name="zubereitung">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="schritt" type="xs:string" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>

<xs:element name="zutat">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="ingredienz" type="xs:string"/>
      <xs:element name="menge" type="xs:decimal"/>
      <xs:element name="einheit" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
</xs:schema>
```

XSDs einbinden



```
<rootElement xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
              xsi:noNamespaceSchemaLocation="schema.xsd">
```

...

```
</rootElement>
```

```
<rezept xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="https://www.hochleitner.cc/schemas/rezept.xsd"
        quelle="https://www.ichkoche.at/der-klassische-tiroler-knoedel-rezept-4386">
```

...

```
</rezept>
```


XML-

Anwendungen

n



Mittels XML definierte Markupsprachen.



Ein paar Beispiele

SVG

Scalable Vector Graphics, ein populäres **Vektorgrafikformat**.

MathML

Ein Dokumentenformat zur Beschreibung **mathematischer Ausdrücke**.

XHTML

HTML 4.01 in XML formuliert, seit 2018 superseded.

SMIL

Synchronized Multimedia Integration Language, ein **Format für zeitbasierte, multimediale Inhalte**.

RDF

Resource Description Framework, eine Sprache zur **Beschreibung von Informationen** über Objekte.

RSS

Really Simple Syndication, ein Format zur Veröffentlichung von **Änderungen auf Websites**.

JSON

JavaScript Object Notation



Was **JSON** ist...

Kompaktes Datenformat

Ein Subset der
Objekt-Literal-Notation von
JavaScript (= JS-Objekte).

Sprachunabhängig. Verwandt mit
JS, überall einsetzbar

Populär im Datenaustausch

Dateien haben die Endung **.json**.

Weit verbreitet für die
Kommunikation im Web (z.B. REST
APIs)



Zurück zum **Tiroler Knödel** in JSON

Dasselbe Rezept, ein anderes
Datenformat.



Rezept Teil 1



```
{  
  "gericht": "Der klassische Tiroler Knödel",  
  "quelle": "https://www.ichkoche.at/der-klassische-tiroler-knoedel-rezept-4386",  
  "zutaten": [  
    {  
      "ingredienz": "Semmelwürfel",  
      "menge": 350,  
      "einheit": "g"  
    },  
    {  
      "ingredienz": "Milch",  
      "menge": 200,  
      "einheit": "ml"  
    },  
    ...  
  ],  
  ...  
}
```

Rezept Teil 2



```
"zubereitung": [  
  "Semmelwürfel mit Milch und Eiern vermengen und 20 bis 30 Minuten ziehen lassen.",  
  "Kleinwürfelig geschnittenen Speck und fein gewiegte Bergsteigerwurst in  
  zerlassener Butter gemeinsam mit Zwiebeln, Petersilie und Schnittlauch anbraten  
  und mit Salz und Muskatnuss abschmecken. Mehl darüber streuen und aus allen  
  Zutaten eine feste, aber nicht zu feuchte Masse kneten.",  
  "Mit befeuchteten Händen daraus Knödel formen.",  
  "Einen davon als Probeknödel in Salzwasser knappe 15 Minuten kochen. Hat der  
  Probeknödel eine zu weiche Konsistenz, muss man noch etwas Mehl unter die  
  Knödelmasse kneten.",  
  "Erst wenn die Masse kompakt genug ist, kocht man die restlichen Knödel.",  
  "Tiroler Knödel mit Schnittlauch bestreut servieren."  
]  
}
```



Zwei

Datenstrukturen

Objekte

```
{  
  name1: wert1,  
  name2: wert2  
}
```

Arrays

```
[  
  wert1,  
  wert2,  
  wert3  
]
```




Vier einfache Werttypen (Literals)

String & Number

`"Semmelwürfel"`

`350`

true & false

`true`

`false`

null

`null`

Objekte



```
{ name1: wert1, name2: wert2 }
```

```
{  
  name1: wert1,  
  name2: wert2  
}
```

```
{  
  "ingredienz": "Semmelwürfel",  
  "menge": 350,  
  "einheit": "g"  
}
```

Arrays



```
[ wert1, wert2, wert3 ]
```

```
[  
  wert1,  
  wert2,  
  wert3  
]
```

```
"zubereitung": [  
  "Semmelwürfel mit Milch ...",  
  "Kleinwürfelig geschnittenen Speck ...",  
  "Mit befeuchteten Händen ...",  
  "Einen davon als Probeknödel ...",  
  "Erst wenn die Masse kompakt genug ist, ...",  
  "Tiroler Knödel mit Schnittlauch bestreut servieren."  
]
```

Literale



```
{  
  "gericht": "Der klassische Tiroler Knödel"  
}
```

```
{  
  "menge": 350  
}
```

```
{  
  "aus_österreich": true,  
  "vegetarisch": false,  
  "bewertung": null  
}
```

JSON Schema

JSON-Dokumente validieren





Schema-Aufbau

Textdokument (Erweiterung `.schema.json`) mit Deklarationen im JSON-Format:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://www.hochleitner.cc/schemas/rezept.schema.json",
  "title": "Rezept",
  "description": "Ein Kochrezept mit Zutaten und Zubereitungsschritten.",
  "type": "object",
  "properties": { ... },
  "additionalProperties": false,
  "required": [ ... ]
}
```



Eigenschaften deklarieren

```
properties: {  
  "gericht": {  
    "description": "Das Gericht oder der Name des Rezepts.",  
    "type": "string",  
    "minLength": 1  
  },  
  "quelle": {  
    "description": "Die Quelle oder URL des Rezepts.",  
    "type": "string",  
    "format": "uri"  
  },  
}
```

Verschachtelte

Properties



```
"zutaten": {  
  "description": "Die Liste der Zutaten für das Rezept.",  
  "type": "array",  
  "minItems": 1,  
  "items": {  
    "type": "object",  
    "properties": {  
      "ingredienz": {  
        "description": "Eine einzelne Zutat des Rezepts.",  
        "type": "string",  
        "minLength": 1  
      },  
      "menge": {  
        "description": "Die Menge der Zutat.",  
        "type": "number",  
        "exclusiveMinimum": 0  
      },  
      "einheit": {  
        "description": "Die Einheit der Menge.",  
        "type": "string",  
        "enum": [  
          "g",  
          "ml",  
          "Stück",  
          "Prise"  
        ]  
      }  
    }  
  }  
}
```

```
...  
"enum": [  
  "g",  
  "ml",  
  "Stück",  
  "Prise"  
]  
}  
}  
}
```


Verpflichtend & zusätzlich



```
"properties": {  
  "  
  "additionalProperties": false,  
  "required": [  
    "gericht",  
    "quelle",  
    "zutaten",  
    "zubereitung"  
  ]  
}
```

Tiroler Knödel **deklariert**



```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://www.hochleitner.cc/schemas/rezept.schema.json",
  "title": "Rezept",
  "description": "Ein Kochrezept mit Zutaten und Zubereitungsschritten.",
  "type": "object",
  "properties": {
    "gericht": {
      "description": "Das Gericht oder der Name des Rezepts.",
      "type": "string",
      "minLength": 1
    },
    "quelle": {
      "description": "Die Quelle oder URL des Rezepts.",
      "type": "string",
      "format": "uri"
    }
  }
}
```

Tiroler Knödel **deklariert**



```
"zutaten": {
  "description": "Die Liste der Zutaten für das Rezept.",
  "type": "array",
  "minItems": 1,
  "items": {
    "type": "object",
    "properties": {
      "ingredienz": {
        "description": "Eine einzelne Zutat des Rezepts.",
        "type": "string",
        "minLength": 1
      },
      "menge": {
        "description": "Die Menge der Zutat.",
        "type": "number",
        "exclusiveMinimum": 0
      },
      "einheit": {
        "description": "Die Einheit der Menge.",
        "type": "string",
        "enum": [
          "g", "ml", "Stück", "Prise"
        ]
      }
    }
  }
},
```

Tiroler Knödel **deklariert**



```
{
  "additionalProperties": false,
  "required": [
    "ingredienz",
    "menge",
    "einheit"
  ],
  "zubereitung": {
    "description": "Die Zubereitungsschritte des Rezepts.",
    "type": "array",
    "minItems": 1,
    "items": {
      "type": "string",
      "minLength": 1
    }
  }
},
"additionalProperties": false,
"required": [
  "gericht",
  "quelle",
  "zutaten",
  "zubereitung"
]
}
```

Schema **anwenden**



Schema kann **nicht** in der JSON-Datei angegeben werden.

Muss von Tools (IDE, Validatoren etc.) geprüft werden.

Z.B. In PhpStorm:

File → Settings → Schemas and DTDs → JSON Schema Mappings

Öffentliche Schemas im JSON Schema Store (<https://www.schemastore.org/>)

YAML

YAML Ain't Markup Language



Was **YAML** ist...

Kompakte(re)s Datenformat

Eine **Übermenge** von JSON.

Jedes **JSON-Dokument** ist auch ein gültiges **YAML-Dokument** (aber nicht umgekehrt).

Populär für Konfigurationen

Dateien haben die **Endung** **.yaml** (oder **.yml**).

Weit verbreitet für die **Speicherung von Konfigurationsdateien** (z.B. in Frameworks).



Finally: **Tiroler Knödel** in YAML

Und noch ein drittes Mal!



Rezept Teil 1



```
# Tiroler Knödel Rezept
```

```
%YAML 1.2
```

```
---
```

```
gericht: Der klassische Tiroler Knödel
```

```
quelle: https://www.ichkoche.at/der-klassische-tiroler-knoedel-rezept-4386
```

```
zutaten:
```

- ingredienz: Knödelbrot
menge: 350
einheit: g
- ingredienz: Milch
menge: 200
einheit: ml
- ingredienz: Eier
menge: 4
einheit: Stück
- ingredienz: Speck
menge: 100
einheit: g

```
# ...
```

Rezept Teil 2



zubereitung:

- >
Semmelwürfel mit Milch und Eiern vermengen und 20 bis 30 Minuten ziehen lassen.
- >
Kleinwürfelig geschnittenen Speck und fein gewiegte Bergsteigerwurst in zerlassener Butter gemeinsam mit Zwiebeln, Petersilie und Schnittlauch anbraten und mit Salz und Muskatnuss abschmecken. Mehl darüber streuen und aus allen Zutaten eine feste, aber nicht zu feuchte Masse kneten.
- >
Mit befeuchteten Händen daraus Knödel formen.
- >
Einen davon als Probeknödel in Salzwasser knappe 15 Minuten kochen. Hat der Probeknödel eine zu weiche Konsistenz, muss man noch etwas Mehl unter die Knödelmasse kneten.
- >
Erst wenn die Masse kompakt genug ist, kocht man die restlichen Knödel.
- >
Tiroler Knödel mit Schnittlauch bestreut servieren.



Zwei

Datenstrukturen

Mappings (Objekte)

name1: wert1,
name2: wert2

Sequences (Arrays)

- wert1,
- wert2,
- wert3



Vier einfache Werttypen (Skalare)

String & Number

Schmelzwürfel

350

true & false

true

false

null

null

Mappings



Block-Format:

```
name1: wert1  
name2: wert2
```

```
ingredienz: Semmelwürfel  
menge: 350  
einheit: g
```

Flow-Format:

```
{ name1: wert1, name2: wert2 }
```

```
{  
  name1: wert1,  
  name2: wert2  
}
```

```
{ ingredienz:  
  Semmelwürfel, menge:  
  350, einheit: g }
```

Sequences



Block-Format:

- wert1
- wert2
- wert3

- Semmelwürfel mit Milch ...
- Kleinwürfelig geschnittenen Speck ...
- Mit befeuchteten Händen ...
- Einen davon als Probeknödel ...
- Erst wenn die Masse kompakt genug ist ...
- Tiroler Knödel mit Schnittlauch bestreut servieren.

Flow-Format:

[wert1, wert2, wert3]

[Semmelwürfel mit Milch ..., Kleinwürfelig geschnittenen Speck ..., Mit befeuchteten Händen ..., Einen davon als Probeknödel ..., Erst wenn die Masse kompakt genug ist ..., Tiroler Knödel mit Schnittlauch bestreut servieren.]

Skalare



```
gericht: Der klassische Tiroler Knödel  
menge: 360  
vegetarisch: false  
bewertung: null
```

- |
Dieser Text bricht
genauso um, wie er
hier geschrieben ist.
- >
Dieser Text ist im Prinzip ein langer
String und hier nur zwecks Lesbarkeit auf
mehrere Zeilen aufgeteilt.

Struktur & Direktiven



%YAML 1.2

```
rezept: Tiroler Knödel  
schwierigkeit: einfach  
vegetarisch: false
```

...

```
rezept: Käsespätzle  
schwierigkeit: einfach  
vegetarisch: true
```

...

Einrückungen



YAML hat keine Klammerung (wie JSON), daher wird in Block-Format eingerückt: 2 oder 4 Leerzeichen (keine Tabulatoren).

zutaten:

- `ingredienz: Semmelwürfel`
 `menge: 350`
 `einheit: g`
- `ingredienz: Milch`
 `menge: 200`
 `einheit: ml`

YAML validieren

Mit JSON-Schema.

Die IDE/das Tool übernimmt die Prüfung.





Nächstes Mal in WEF1VO

JavaScript-Grundlagen

Credits



- S. 4: Apollo 11 Saturn V about to clear the tower: <https://history.nasa.gov/alsj/a11/images11.html#Launch>
- S. 5: Buzz has both feet on the footpad. His hands are between the third and fourth rungs. His bent knees suggest that he is about to try to jump up to the lowest rung. His OPS antenna is up: <https://history.nasa.gov/alsj/a11/images11.html#Mission>
- S. 6: CapCom Charlie Duke (left), backup Commander Jim Lovell (next right), and backup Lunar Module Pilot Fred Haise (next to Lovell) in the MOCR during the Apollo 11 landing: <https://history.nasa.gov/alsj/a11/images11.html#Support>
- S. 7: The 12-bit CDC 160 and 160-A architecture were the basis of the peripheral processors (PPs) in the CDC 6000 series: https://en.wikipedia.org/wiki/Control_Data_Corporation#/media/File:Control_Data_160-A.jpg
- S. 11/40/59: Der klassische Tiroler Knödel: <https://www.ichkoche.at/der-klassische-tiroler-knoedel-rezept-4386>